DSA
Practical 9

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

CODE:

```
#include <iostream>
using namespace std;

int q[20], front = -1, rear = -1;
int stackArr[20], top = -1;
int a[20][20], vis[20];

// ---------------- QUEUE FUNCTIONS (for BFS)
void add(int item) {
```

```
    if (rear == 19)
        cout << "Queue Full\n";
    else {
        if (rear == -1) {
            rear++;
            front = 0;
        } else {
            rear++;
        }
        q[rear] = item;
    }
}

int removeQ() {
    if (front == -1 || front > rear)
        return 0;
    return q[front++];
}

// ---------------- STACK FUNCTIONS (for DFS)
void push(int item) {
    if (top == 19)
```

```
        cout << "Stack Overflow\n";
    else
        stackArr[++top] = item;
}

int pop() {
    if (top == -1)
        return 0;
    return stackArr[top--];
}

// ---------------- BFS ALGORITHM
void bfs(int s, int n) {
    int p, i;
    add(s);
    vis[s] = 1;
    p = removeQ();
    if (p != 0) cout << p << " ";
    while (p != 0) {
        for (i = 1; i <= n; i++) {
            if (a[p][i] != 0 && vis[i] == 0) {
                add(i);
```

```
            vis[i] = 1;
        }
    }
    p = removeQ();
    if (p != 0) cout << p << " ";
  }
  // Check unvisited nodes
  for (i = 1; i <= n; i++)
    if (vis[i] == 0)
      bfs(i, n);
}

// ---------------- DFS ALGORITHM
void dfs(int s, int n) {
  int k, i;
  push(s);
  vis[s] = 1;
  k = pop();
  if (k != 0) cout << k << " ";
  while (k != 0) {
    for (i = 1; i <= n; i++) {
      if (a[k][i] != 0 && vis[i] == 0) {
```

```
            push(i);
            vis[i] = 1;
        }
    }
    k = pop();
    if (k != 0) cout << k << " ";
  }
  for (i = 1; i <= n; i++)
    if (vis[i] == 0)
      dfs(i, n);
}

// ---------------- MAIN FUNCTION
int main() {
  int n, i, j, s, ch;
  char c;
  cout << "Enter number of vertices: ";
  cin >> n;

  // INPUT ADJACENCY MATRIX
  for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j++) {
```

```
            cout << "Enter 1 if " << i << " connected to " << j << "
else 0: ";
            cin >> a[i][j];
        }
    }

    cout << "Adjacency Matrix:\n";
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++)
            cout << a[i][j] << " ";
        cout << endl;
    }

    do {
        // reset visited array
        for (i = 1; i <= n; i++)
            vis[i] = 0;

        cout << "\nMenu\n1. BFS\n2. DFS\nEnter your choice: ";
        cin >> ch;
        cout << "Enter source vertex: ";
        cin >> s;
```

```
        if (ch == 1)
            bfs(s, n);
        else
            dfs(s, n);

        cout << "\nDo you want to continue (Y/N)? ";
        cin >> c;
    } while (c == 'Y' || c == 'y');

    return 0;
}
```

OUTPUT

```
Enter number of vertices: 3
Enter 1 if 1 connected to 1 else 0: 1
Enter 1 if 1 connected to 2 else 0: 2
Enter 1 if 1 connected to 3 else 0:
3
Enter 1 if 2 connected to 1 else 0: 3
Enter 1 if 2 connected to 2 else 0: 4
Enter 1 if 2 connected to 3 else 0: 5
Enter 1 if 3 connected to 1 else 0:
6
Enter 1 if 3 connected to 2 else 0: 7
Enter 1 if 3 connected to 3 else 0: 8
Adjacency Matrix:
1 2 3
3 4 5
6 7 8

Menu
1. BFS
2. DFS
Enter your choice: 1
Enter source vertex: 2
2 1 3
```